

Assignment #4

2023480023 정태양

Problem 1: Explore the α, β -Crown Models Directory

Models:

complete_verifier/models 하위에는 alpha-beta-CROWN에서 제공하는 검증용 benchmark 모델들이 포함되어있음. 모델들은 주로 MNIST, CIFAR-10, CIFAR-100, OVAL, ERAN, SDP benchmark 계열로 구성되어 있으며, 단순한 fully-connected network부터 CNN, ResNet 계열 모델까지 포함되어있음. 모델 포맷은 대부분 PyTorch checkpoint인 .pth 형태이고, 일부 benchmark에서는 .model, .pkl, .pt 파일도 사용됨. 예를 들어 cifar10_resnet/에는 CIFAR-10 ResNet 계열 모델인 resnet2b.pth, resnet4b.pth와 해당 구조를 정의하는 resnet.py가 포함되어있고, eran/에는 MNIST/CIFAR용 MLP, CNN, ResNet 모델들이 포함되어있음. non_relu/에는 sigmoid activation을 사용하는 모델도 있어 ReLU 이외의 activation을 포함한 검증 예제도 확인할 수 있음.

Verification query examples:

complete_verifier/exp_configs 하위에는 alpha-beta-CROWN을 실행하기 위한 다양한 YAML 예제들이 포함되어있음. tutorial_examples/에는 PyTorch model, ONNX model, custom model, custom dataloader, VNNLIB specification 등을 사용하는 기본 예제들이 있고, beta_crown/에는 MNIST, CIFAR, ResNet, OVAL 모델에 대한 일반적인 beta-CROWN 검증 설정들이 있음. GCP-CROWN/과 BICCOS/에는 cutting-plane 또는 개선된 bound propagation 설정들이 포함되어있으며, bab_attack/에는 branch-and-bound 기반으로 counterexample을 찾는 설정들이 있음. 또한 vnncomp21/부터 vnncomp25/까지는 VNN-COMP benchmark용 설정들이 정리되어있어 ACASXu, TinyImageNet, CIFAR-100, control/RL, ViT, YOLO 계열 benchmark 등 다양한 verification setup을 확인할 수 있음.

Difference from Marabou:

Marabou는 solver query를 직접 작성하는 느낌이 강하고, alpha-beta-CROWN은 이미 정해진 configuration system 안에서 모델과 검증 조건을 조합하는 방식에 가까움. 결과 표현도 Marabou의 SAT/UNSAT 중심 해석과 달리, alpha-beta-CROWN에서는 verified, falsified, timeout으로 결과를 정리하는 방식이 일반적임.

Problem 2: Run α, β -Crown with an External Model and Dataset

설치 확인을 위해 제공된 tutorial config 중 custom_box_data_example.yaml을 실행하였고, returncode=0으로 정상 종료되는 것을 확인하였다. 외부 모델로는 sklearn digits 데이터셋으로 직접 학습한 작은 CNN을 사용하였다. 이 모델은 Conv2d-ReLU 2층과 Linear-ReLU-Linear 분류기로 구성되어 있으며, 입력 크기가 1x8x8로 작아 complete verification 실험에 적합하다고 판단하였다. 또한 HW3에서 Marabou로 검증했던 모델과 같은 구조를 사용하여 이후 비교가 가능하도록 하였다. 입력은 [0,1] 범위로 정규화하였다. 검증 설정은 digits_linf.yaml에 작성하였다. 검증 property는 선택된 입력에 대해 L_∞ 거리 $\epsilon=0.01$ 안의 모든 입력이 원래 class를 유지하는지 확인하는 방식으로 설정하였다. 모델은 테스트셋에서 약 95.83%의 정확도를 보였고, 검증 대상 샘플은 index=348, label=6이었다. α, β -CROWN 실행 결과는 safe-incomplete였으며, 이는 과제 기준으로 Verified로 해석하였다. 결과는 verified=1, falsified=0, timeout=0이었고, 검증 시간은 약 9.98초, 전체 실행 시간은 약 16.45초였다. 반례는 발견되지 않았다.

Report

본 실험에서는 Assignment #3에서 Marabou로 검증했던 것과 같은 sklearn digits 기반의 작은 CNN을 α, β -CROWN에 적용하였다. 모델은 Conv2d-ReLU 2개 층과 Linear-ReLU-Linear 분류기로 구성되며, 입력 크기는 1x8x8이다. 처음부터 큰 모델을 사용하기보다 작은 모델을 선택한 이유는 complete verification에서 모델 크기와 입력 차원이 검증 시간에 큰 영향을 주기 때문이다. 또한 같은 모델을 사용하면 Marabou와 α, β -CROWN의 차이를 단순히 모델 차이가 아니라 verifier 자체의 차이로 비교할 수 있다고 판단하였다. 데이터셋은 sklearn.datasets.load_digits를 사용하였고, 0부터 16 사이의 grayscale값을 [0,1] 범위로 정규화하였다. 검증 대상은 테스트셋에서 올바르게 분류된 sample_index=348로 선택하였다. 해당 샘플의 실제 label과 모델 예측은 모두 6이었고, 전체 테스트 정확도는 약 95.83%였다. 검증 property는 이 입력 주변의 L_∞ 반경 $\epsilon=0.01$ 안에 있는 모든 입력이 원래 class 6으로 계속 분류되는지 확인하는 방식으로 설정하였다. α, β -CROWN 실행 결과는 safe-incomplete였고, 로그에서는 verified=1, falsified=0, timeout=0으로 확인되었다. 따라서 과제 기준으로는 해당 샘플이 $\epsilon=0.01$ 범위에서 Verified되었다고 해석하였다. 검증 내부 시간은 약 9.98초, 전체 실행 시간은 약 16.45초였다. 반례는 발견되지 않았다.

흥미로웠던 점은 결과 이름이 단순히 safe가 아니라 safe-incomplete였다는 점이다. 처음에는 incomplete라는 표현 때문에 검증이 완전히 끝나지 않은 것으로 보였지만, 로그에서 verified count가 1로 기록되어 있었고 bab_ret이 비어 있었다. 이를 통해 branch-and-bound까지 가지 않고 초기 bound propagation 단계에서 이미 안전하다고 판단된 결과로 이해하였다. Marabou와 비교하면, 같은 모델과 같은 sample, 같은 $\epsilon=0.01$ 조건에서 Marabou는 모든 target class에

대해 UNSAT을 반환했고 각 target 검증 시간은 약 0.01~0.02초였다. 반면 α, β -CROWN은 하나의 robustness query로 처리했지만 내부 검증 시간은 약 9.98초로 더 길었다. 이 결과만 보면 Marabou가 더 빠른 것처럼 보이지만, 이번 실험은 입력이 8x8인 매우 작은 모델 하나만 대상으로 했기 때문에 α, β -CROWN의 확장성이 잘 드러나는 상황은 아니었다고 판단된다. 오히려 작은 문제에서는 verifier 초기화, YAML 설정 해석, PyTorch 모델 로딩 같은 고정 비용이 상대적으로 크게 반영된 것으로 보인다. 사용성 측면에서는 Marabou가 더 직관적이었다. Marabou는 Python API에서 입력 bound와 출력 제약을 직접 추가하므로, 내가 어떤 property를 검증하는지 코드 흐름에서 바로 확인할 수 있었다. 반면 α, β -CROWN은 YAML 파일, PyTorch 모델 loader, custom data loader가 함께 맞아야 실행되기 때문에 처음에는 구조를 파악하는 데 시간이 더 걸렸다. 특히 Customized("test.py", ...) 방식으로 모델과 데이터를 불러오는 부분, 그리고 결과가 .pkl과 stdout log에 나뉘어 저장되는 방식은 처음에는 다소 낯설었다. 하지만 α, β -CROWN은 모델, 데이터셋, perturbation 크기, solver 설정을 YAML에 분리해두면 같은 코드로 여러 설정을 반복 실험하기 쉬웠다. 또한 bound propagation과 branch-and-bound를 결합한 구조이기 때문에, 더 큰 네트워크에서는 Marabou보다 유리할 가능성이 있다고 생각한다. 이번 실험에서는 작은 모델이라 속도상의 장점은 확인하지 못했지만, 설정 파일 기반의 재현성과 solver option을 조정할 수 있다는 점은 α, β -CROWN의 강점으로 보였다.